

Descreva o processo de desenvolvimento de uma app móvel que funciona como calculadora.



O desenvolvimento de uma aplicação móvel que funciona como uma calculadora envolve várias etapas, desde o planejamento inicial até o lançamento e manutenção. Aqui está uma descrição detalhada do processo:

1. Planejamento

a. Definição do Escopo

- **Funcionalidades Básicas:** Operações aritméticas simples (adição, subtração, multiplicação, divisão).
- **Funcionalidades Avançadas:** Operações científicas (exponenciação, logaritmos, funções trigonométricas), histórico de cálculos, modos de exibição (paisagem, retrato).

b. Pesquisa de Mercado

- **Concorrência:** Analisar outras calculadoras disponíveis no mercado.
- **Público-Alvo:** Determinar quem são os usuários finais (estudantes, profissionais, público geral).

c. Requisitos

- **Funcionais:** Lista de todas as funcionalidades que a aplicação deve ter.
- **Não Funcionais:** Performance, segurança, compatibilidade com diferentes dispositivos e sistemas operacionais.

2. Design

a. Wireframes e Mockups

- **Wireframes:** Esboços básicos das telas principais da aplicação.
- **Mockups:** Designs de alta fidelidade que mostram a aparência final das telas.

b. Experiência do Usuário (UX)

- **Fluxo de Usuário:** Definição de como os usuários navegarão pela aplicação.
- **Usabilidade:** Garantir que a aplicação seja fácil e intuitiva de usar.

c. Interface do Usuário (UI)

- **Estilo Visual:** Escolha de cores, tipografia, ícones e outros elementos visuais.
- **Responsividade:** Design adaptável para diferentes tamanhos de tela.

3. Desenvolvimento

a. Escolha da Tecnologia

- **Plataforma:** Decidir entre desenvolvimento nativo (Swift para iOS, Kotlin/Java para Android) ou multiplataforma (React Native, Flutter).

b. Configuração do Ambiente de Desenvolvimento

- **Ferramentas:** IDEs, emuladores/simuladores, bibliotecas e frameworks necessários.

c. Desenvolvimento Front-End

- **Interface:** Implementação das telas e componentes de interface.
- **Interações:** Programação das interações e navegação entre telas.

d. Desenvolvimento Back-End

- **Lógica de Cálculo:** Implementação das funcionalidades de cálculo.

- **Persistência de Dados:** Se necessário, para histórico de cálculos ou configurações do usuário.

e. Integração

- **APIs:** Se houver integração com serviços externos, como conversores de moedas ou unidades.

4. Testes

a. Testes Unitários

- **Cálculos:** Garantir que todas as operações aritméticas e científicas funcionem corretamente.

b. Testes de Interface

- **UI/UX:** Verificar se todos os componentes da interface funcionam conforme esperado e são usáveis.

c. Testes de Performance

- **Responsividade:** Garantir que a aplicação seja rápida e eficiente.

d. Testes de Compatibilidade

- **Dispositivos:** Testar em diferentes dispositivos e versões de sistemas operacionais.

5. Lançamento

a. Preparação para o Lançamento

- **Metadados:** Títulos, descrições, capturas de tela e vídeos para as lojas de aplicativos.
- **Políticas:** Políticas de privacidade e termos de uso.

b. Publicação

- **Lojas de Aplicativos:** Submeter a aplicação para a App Store (iOS) e Google Play Store (Android).

6. Manutenção

a. Monitoramento

- **Feedback:** Coleta de feedback dos usuários para melhorias contínuas.
- **Erros:** Monitoramento de erros e falhas para correções rápidas.

b. Atualizações

- **Funcionalidades:** Adição de novas funcionalidades e melhorias.
- **Correções:** Correção de bugs e problemas de compatibilidade.

c. Suporte ao Usuário

- **Documentação:** Manuais de uso e FAQs.
- **Suporte Técnico:** Atendimento aos usuários para resolução de problemas.

7. Iteração Contínua

- **Ciclo de Vida:** A aplicação deve passar por um ciclo contínuo de melhorias, baseado no feedback e nas novas tecnologias disponíveis.

Seguir esses passos garante que a aplicação de calculadora será bem projetada, funcional, e atenderá às necessidades dos usuários de maneira eficaz e eficiente.

Uma calculadora envolve receber números e realizar operações. Uma sugestão de resolução é perceber a que opções o utilizador tem acesso.

O utilizador pode apenas inserir números e operações.

- Como e em que ordem é que o programa deve receber estas instruções? Primeiro os números? Primeiro as operações? Pode

realizar várias operações? O que significa eu clicar no ponto enquanto estou a digitar um número?



Para desenvolver uma calculadora, é essencial planejar cuidadosamente como o programa receberá e processará as instruções do usuário. Vamos explorar as questões que você levantou, fornecendo uma estrutura clara para o funcionamento da calculadora.

Ordem de Recebimento das Instruções

1. Entrada Inicial (Primeiros Números e Operações)

- **Primeiro os Números:** O usuário começa inserindo um número. Este número é armazenado como o primeiro operando.
- **Operações:** Após inserir o primeiro número, o usuário pode selecionar uma operação (adição, subtração, multiplicação, divisão, etc.).
- **Segundo Número:** Após a operação, o usuário insere o próximo número, que será o segundo operando.

2. Fluxo de Entrada de Dados

- **Número -> Operação -> Número:** Este é o fluxo básico que se repete. O usuário insere um número, escolhe uma operação e insere o próximo número.
- **Sequência de Operações:** A calculadora deve suportar a inserção contínua de números e operações. Por exemplo, para calcular $5 + 3 * 2$, a calculadora deve permitir a entrada de **5**, depois **+**, em seguida **3**, depois *****, e finalmente **2**.

Realização de Várias Operações

- **Operações Sequenciais:** A calculadora deve permitir a realização de operações sequenciais. Isso pode ser feito armazenando o resultado de uma operação intermediária como o novo operando para a próxima operação.
- **Prioridade das Operações:** Implementar a prioridade das operações (PEMDAS/BODMAS) para garantir cálculos corretos. Por exemplo, na sequência $5 + 3 * 2$, a multiplicação deve ser realizada antes da adição.

Tratamento de Ponto Decimal

- **Inserção do Ponto:** Quando o usuário clica no ponto (.), a calculadora deve permitir a inserção de números decimais.
- **Formato Correto:** O ponto só pode ser inserido uma vez em um número. O programa deve impedir múltiplos pontos no mesmo número (por exemplo, **3.14** é válido, mas **3.14.5** não é).

Estrutura do Programa

Vamos delinear a lógica principal do programa:

1. Inicialização

- **Variáveis:** Inicializar variáveis para armazenar o primeiro número, o operador e o segundo número.
- **Estado:** Manter um estado para rastrear se a entrada atual é um número ou uma operação.

2. Entrada do Usuário

- **Números:** Se a entrada for um número, verificar se é parte do primeiro ou do segundo número, conforme o estado atual.
- **Operações:** Se a entrada for uma operação, armazenar o operador e mudar o estado para receber o próximo número.

3. Processamento das Operações

- **Operação Imediata:** Se o usuário pressiona o botão de igual (=), calcular o resultado imediato entre o primeiro número, o operador e o segundo número.
- **Operações Contínuas:** Se uma nova operação é inserida após uma sequência (como **5 + 3 ***), calcular o resultado intermediário e usar esse resultado como o primeiro operando para a próxima operação.

4. Exibição

- **Atualização da Tela:** Após cada entrada, atualizar a tela para mostrar o número atual, a operação escolhida e o resultado intermediário, se aplicável.

Exemplo de Implementação em Pseudo-Código

pseudo

 Copiar código

```

Atualizar: primeiro_numero = "" segundo_numero = "" operador = "" estado = "esperando_primeiro_numero" resultado = 0
entrada_usuario(dado): se dado é número: se estado é "esperando_primeiro_numero" ou "digitando_primeiro_numero":
adicionar dado a primeiro_numero estado = "digitando_primeiro_numero" se estado é "esperando_segundo_numero" ou "digitando_segundo_numero":
adicionar dado a segundo_numero estado = "digitando_segundo_numero" se dado é ponto_decimal: se estado é "digitando_primeiro_numero" e ponto_decimal não está em primeiro_numero:
adicionar ponto_decimal a primeiro_numero se estado é "digitando_segundo_numero" e ponto_decimal não está em segundo_numero:
adicionar ponto_decimal a segundo_numero se dado é operador: se estado é "digitando_primeiro_numero": operador = dado estado = "esperando_segundo_numero"
se dado é igual: se estado é "digitando_segundo_numero": resultado = calcular(primeiro_numero, operador, segundo_numero)
exibir resultado primeiro_numero = resultado segundo_numero = "" operador = "" estado = "esperando_operador"
calcular(num1, operador, num2): se operador é "+": retornar num1 + num2 se operador é "-": retornar num1 - num2 se operador é "*": retornar num1 * num2
se operador é "/": retornar num1 / num2 exibir(dado): mostrar dado na tela
  
```

Este pseudo-código ilustra o fluxo básico de como a calculadora pode receber números e operações, processar cálculos e exibir resultados. Adicionalmente, a implementação pode ser refinada para incluir funcionalidades mais avançadas e melhorar a usabilidade.